VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Assignment 1 Report

Foundations of Deep Learning Pipelines and Architectures

Advisor(s): Dr. Lê Thành Sách
Student(s): Cao Đức Thành 2453164
Đặng Thành Phú 2452967

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Problem and Data Description **5**
1.1 Problem Formulation . . . 5
1.2 Dataset Description . . . 5
1.3 Exploratory Data Analysis . . . 6
1.4 Data Splitting . . . 7
1.5 Preprocessing and Augmentation . . . 7

2 Methodology **9**
2.1 Pipeline Overview . . . 9
2.1.1 Dataset Partitioning and Preprocessing . . . 10
2.1.2 Data Loading . . . 10
2.1.3 Training and Optimization . . . 10
2.1.4 Prediction, Model Selection, and Evaluation . . . 11
2.2 Linear / Softmax Classifier . . . 12
2.3 Multilayer Perceptron . . . 12
2.4 Convolutional Neural Network . . . 12
2.5 Long Short-Term Memory (LSTM) . . . 13
2.6 Transformer . . . 14
2.7 Training Setup . . . 14
2.7.1 Training Configuration . . . 14
2.7.2 Checkpoint Selection . . . 16
2.7.3 Hyperparameter Selection . . . 16
2.8 Experimental Design . . . 16

3 Implementation Results **17**
3.1 Training Behavior . . . 17
3.1.1 Linear Classifier . . . 18
3.1.2 Multilayer Perceptron . . . 18
3.1.3 Convolutional Neural Network . . . 18
3.1.4 Long Short-Term Memory . . . 18
3.1.5 Transformer . . . 19
3.2 Quantitative Results . . . 19
3.2.1 Main Model Comparison . . . 19
3.2.2 Training and Inference Time . . . 20
3.2.3 Additional Transformer Experiment . . . 20

3.3 Qualitative Results 20
3.4 Comparison and Discussion 23
3.5 Error Analysis 24
3.6 Limitations and Conclusion 24
3.6.1 Dataset Limitations 24
3.6.2 Preprocessing Trade-off: Evaluation Consistency vs. Distribution Adaptation 25
3.6.3 Architectural and Computational Limitations 25
3.6.4 Experimental and Reproducibility Limitations 25
3.6.5 Conclusion 25

4 AI Usage Disclosure **26**

References **27**

A Appendix **27**

1 Problem and Data Description

1.1 Problem Formulation

Real-world motivation: The aim of the assignment is to compare 5 models in recognizing and categorizing the images of fashion to see which models are good at computer vision.

The input: The input of the assignment is Fashion-MNIST image tensor 1x28x28, the pixel value is in range 0 to 255.

The output: the output is the categorical class index y in range 0 to 9 (corresponding to one of 10 mutually exclusive fashion categories).

The prediction unit: The prediction unit is a single number that represents each category.

Softmax function: when forwarding the neural networks, we will have a raw prediction number, to make it the number for classification, we must pass it through a softmax function:

$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{with } i = 1, \ldots, K$

The Cross-Entropy loss: when we have a prediction, we will compare it to our real data to calculate the loss for backpropagation:

$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$

Main difficulties: - Flattening 2D images into 1D vectors (Linear, MLP) or scanning line-by-line(LSTM) destroys the relationship of 2D geometric. - Inter-class visual ambiguity: the upper-body clothes share similar overall outer silhouettes at a low spatial resolution.

1.2 Dataset Description

Dataset source and identity: Fashion-MNIST, released by Zalando Research under the MIT License.

Sample count: 70000 grayscale images of size 28 x 28 accross 10 balanced classes: (0) T-shirt/top, (1) Trouser, (2) Pullover, (3) Dress, (4) Coat, (5) Sandal, (6) Shirt, (7) Sneaker, (8) Bag, (9) Ankle boot.

Storage format: Raw binary byte arrays in compressed IDX format (30 MB).

All clothes in this dataset are already cropped, centered, and placed on a solid black background. In real life, clothes in pictures have cluttered backgrounds, different angles, and varying lighting conditions, so this dataset is much cleaner and simpler than real-world photos.

1.3 Exploratory Data Analysis

The class distribution: the dataset is an ideal uniform distribution: 60000 training images and 10000 testing images.

Looking at the plotted sample images, footwear items (Sandal, Sneaker, Ankle boot) and Trouser have distinct shapes and are easy to tell apart. In contrast, the upper-body clothes (T-shirt, Pullover, Shirt, Coat) share almost the same overall outer shape. At 28 × 28 resolution, small details like collar cuts or buttons are blurry and hard to distinguish.

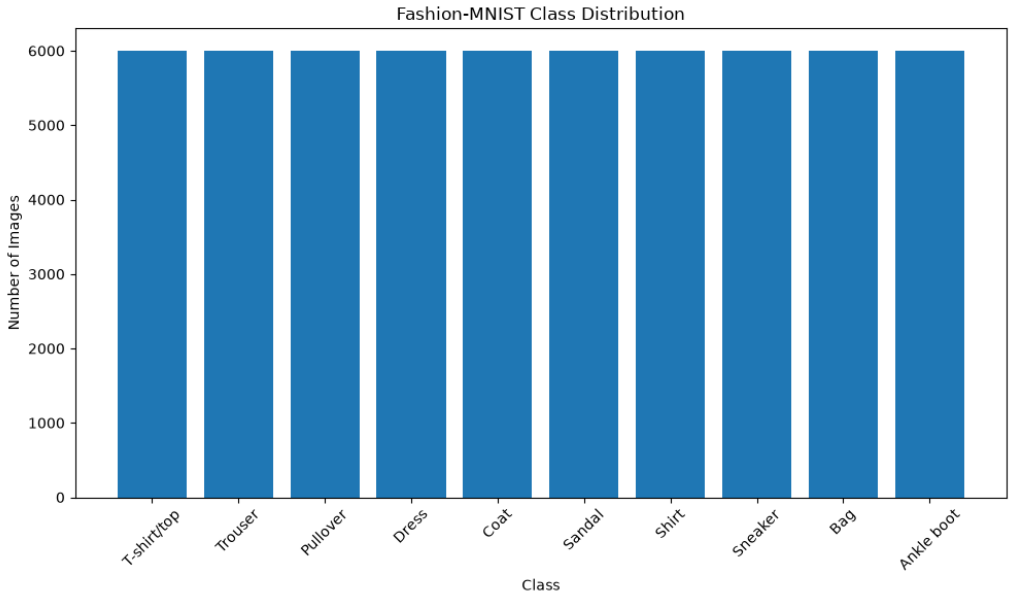


Figure 1.1: Class distribution of the Fashion-MNIST training set.

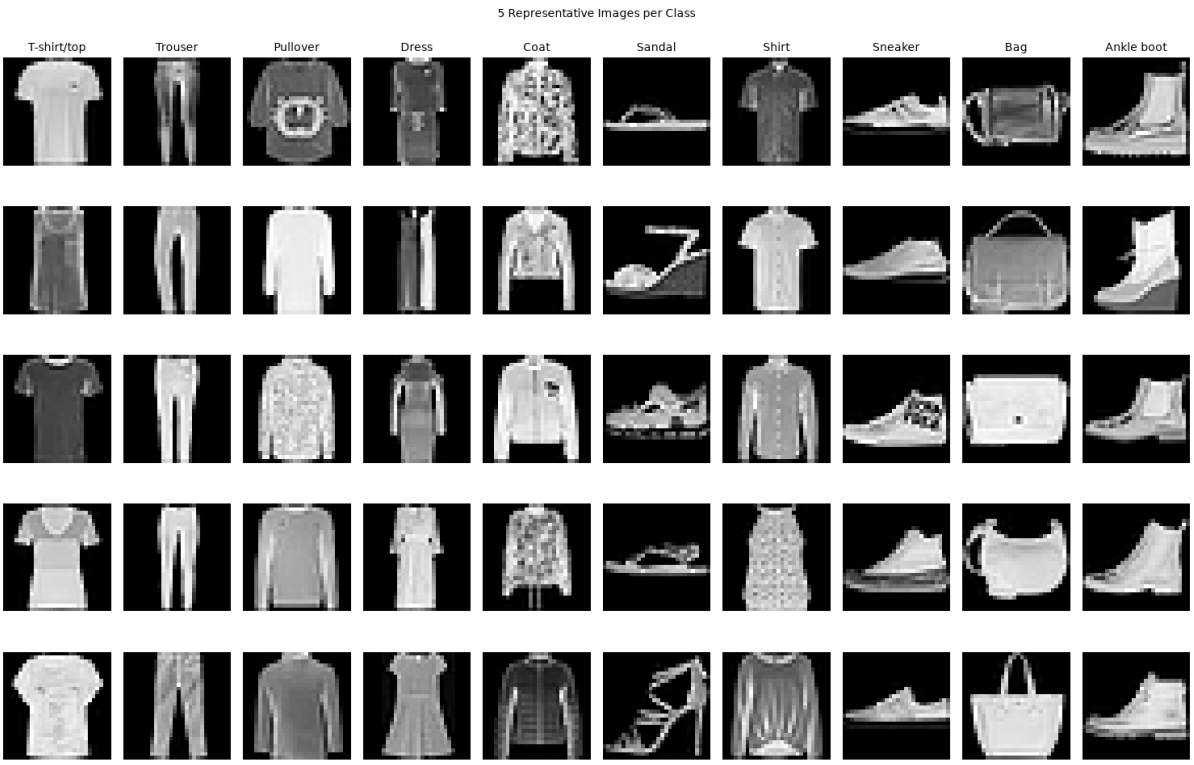


Figure 1.2: Representative Fashion-MNIST images from the ten classes.

1.4 Data Splitting

The 60000 training images were divided into these parts: - Training split: 54000 samples. - Validation split: 6000 samples. - Test split: 10000 samples from dataset

splits were generated using a pytorch random permutation with fixed seed 666. The generated indices were saved in results/split.npx.

Normalizion statics were computed on the 54000 traning samples. Neither validation nor test sets were involved in computing mean or standard deviation, completely eliminating distribution leakage.

1.5 Preprocessing and Augmentation

We transform the data by using transforms.ToTensor() to converts PIL images to pytorch tensors scaled to [0.0, 1.0] and transforms.Normalize() to apply z-score standardization.

We use minibatch size B = 200. Shuffing is enabled for training and disabled for validation/test loaders

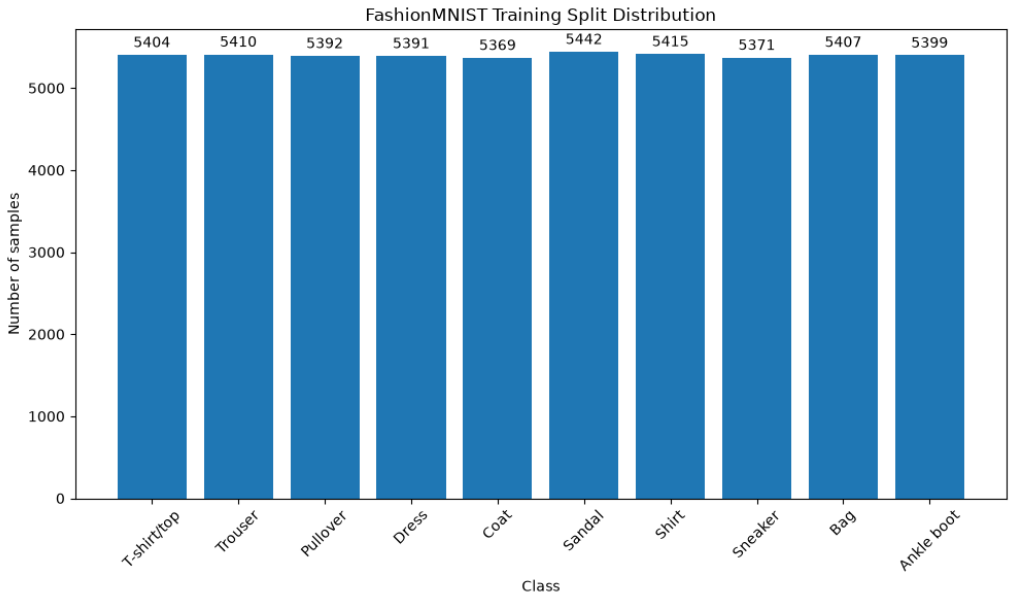


Figure 1.3: Class distribution of the Fashion-MNIST training split after indexing.

Preprocessed batches were verified via `plot_processed_batch()`.

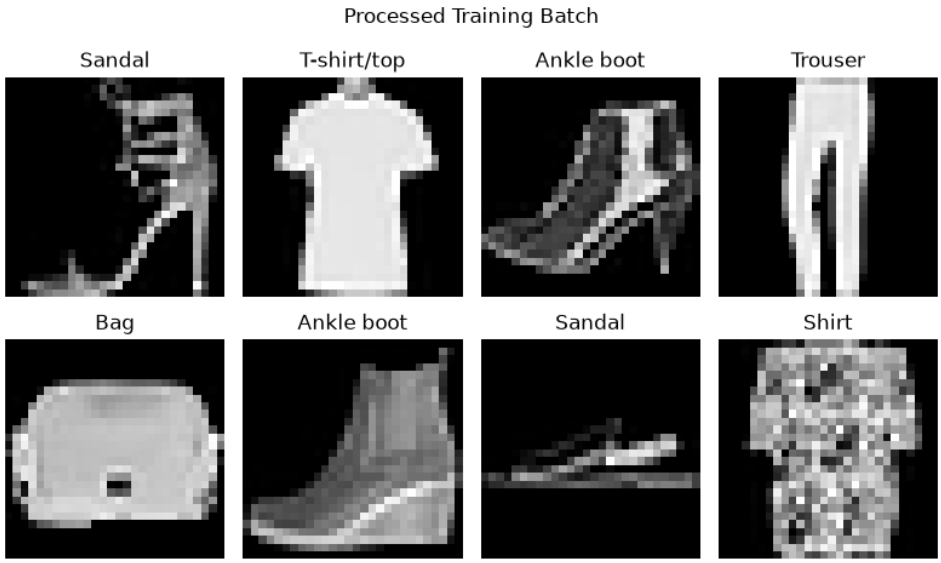


Figure 1.4: Examples from a training batch after preprocessing and normalization.



2 Methodology

2.1 Pipeline Overview

Raw Fashion-MNIST data

↓

Train / validation / test partitioning

↓

Calculate training mean and standard deviation

↓

Preprocessing: `ToTensor()` **+ Standardization**

↓

PyTorch Dataset and DataLoader

↓

Model: Linear / MLP / CNN / LSTM / Transformer

↓

Forward pass and class logits

↓

Cross-entropy loss

↓

Backpropagation and Adam optimizer

↓

Repeat across training batches and epochs

↓

Validation and checkpoint selection

↓

Prediction: $\arg\max$

↓

Final test evaluation

2.1.1 Dataset Partitioning and Preprocessing

Fashion-MNIST contains 70,000 grayscale images of size 28×28 pixels, distributed across ten clothing categories.

The official training partition of 60,000 images is divided into 54,000 training samples and 6,000 validation samples using a reproducible random permutation with seed 666. The official test set contains another 10,000 images.

The generated training and validation indices are saved in `results/split.npz` to preserve the same partition across experiments.

Each images are originally 8 bits pixel values. Then under PyTorch's `ToTensor()`, pixel are scaled into $[0, 1]$.

The normalization statistics are calculated exclusively from the training partition:

$$\begin{aligned} \text{Training mean:} \quad & \mu_{\text{train}} = 0.2859, \\ \text{Training standard deviation:} \quad & \sigma_{\text{train}} = 0.3529. \end{aligned}$$

standardization is performed using:

$$x_{\text{standardized}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{2.1}$$

The same training statistics are applied to training, validation, and test images. This is a trade-off to prevent data leakage.

2.1.2 Data Loading

PyTorch's `Dataset`, `Subset`, and `DataLoader` objects handle the preprocessed data.

Training data are shuffled, while validation and test data are loaded without shuffling.

Each image batch has shape $(B, 1, 28, 28)$, where B denotes the batch size. Meanwhile training time, we use B=200 across all five models, in order to make sure the condition of all five models are the same.

2.1.3 Training and Optimization

Each model generates ten class scores (logits) for each Fashion-MNIST category.

Cross-entropy loss is used to measure classification error:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{i=1}^{B}\log(p_{i,y_i}), \tag{2.2}$$

where p_{i,y_i} denotes the softmax probability assigned to the correct class of sample i, and B is the batch size.

PyTorch's `CrossEntropyLoss` computes the required log-softmax operation internally

Model parameters are updated using the Adam optimizer. For each training batch, the optimization procedure consists of:

1. Resetting previously accumulated gradients.
2. Computing class logits through a forward pass.
3. Calculating cross-entropy loss.
4. Computing gradients through backpropagation.
5. Updating model parameters using Adam.

This process is repeated across training batches and epochs.

2.1.4 Prediction, Model Selection, and Evaluation

The model produces a vector of ten class logits for each input image. The predicted class is obtained by selecting the class with the highest logit.

$$\hat{y} = \arg\max_{k} z_k \tag{2.3}$$

where z_k represents the logit corresponding to class k.

At the end of each epoch, validation performance is measured using loss, accuracy, and macro-F1. The implementation selects the checkpoint that achieves the highest validation macro-F1. In addition, an intermediate checkpoint is saved after every epoch to preserve model states for later inspection.

After training, the selected checkpoint is loaded and evaluated on the test partition. Evaluation outputs include accuracy, macro-F1, parameter counts and training and inference time.

The existing training implementation also evaluates the test partition after every epoch. Although model selection is based on validation macro-F1, this leads to a limitation in developing if considering the strict separation of testing and training, which is discussed in Section 3.6.

2.2 Linear / Softmax Classifier

For the baseline model, we flatten the 28 x 28 image into a 1D vector of 784 numbers. We pass this vector through a single linear layer.

$$z = xW^T + b \tag{2.4}$$

where $W \in \mathbb{R}^{10\times784}$ and $b \in \mathbb{R}^{10}$. In total, the model has $784 \times 10 + 10 = 7,850$ parameters. We compute the loss using PyTorch's `nn.CrossEntropyLoss()` directly on the raw logits z, which internally combines logsoftmax and NLLloss for numerical stability. Since there is no hidden layer or activation function, this model can only create linear decision boundaries, so it cannot learn non-linear relationships between pixels.

2.3 Multilayer Perceptron

To improve upon the linear model, we added a hidden layer with 256 neurons, a ReLU activation, and a dropout layer to reduce overfitting:

$$h = \text{Dropout}(\text{ReLU}(xW_1^T + b_1), p = 0.2), \quad z = hW_2^T + b_2 \tag{2.5}$$

where $W_1 \in \mathbb{R}^{256\times784}$ and $W_2 \in \mathbb{R}^{10\times256}$. This gives $(784 \times 256 + 256) + (256 \times 10 + 10) = 203,530$ parameters. The ReLU activation allows the network to learn non-linear boundaries. However, because we still flatten the image into a 1D vector, the model treats each pixel as an independent number and still ignores the 2D spatial layout of the image.

2.4 Convolutional Neural Network

Unlike Linear and MLP, our CNN keeps the original 2D image shape $[B, 1, 28, 28]$. We designed a CNN with convolutional layers followed by a fully connected classifier. While devoping this model, higher macro-F1 score turns out when MaxPool2d is not included in Convolutional Block 1 and 2.

- **Conv Block 1:** `Conv2d(1, 32, kernel=3, padding=1)` → `ReLU()` (output size: $32 \times 28 \times 28$).
- **Conv Block 2:** `Conv2d(32, 128, kernel=3, padding=1)` → `ReLU()`
- **Conv Block 3:** `Conv2d(128, 128, kernel=3, padding=1)` → `ReLU()` → `MaxPool2d(2, stride=2)` (output size: $128 \times 14 \times 14$).
- **Classifier:** `Flatten()` → `Linear(25088, 128)` → `ReLU()` → `Dropout(0.35)` → `Linear(128, 10)`.



The CNN has $3,397,578$ parameters in total. Most of these parameters (3.21M) come from the first linear layer after flattening $128 \times 14 \times 14 = 25,088$ features. The 3×3 filters slide across the image to scan local patches, which helps the network detect visual features like edges, seams, and textures anywhere in the image.

2.5 Long Short-Term Memory (LSTM)

Unlike CNNs, which process images using spatial filters, our LSTM treats each Fashion-MNIST image as a sequence of horizontal rows. The original input shape $[B, 1, 28, 28]$ is converted into $[B, 28, 28]$, representing 28 time steps with 28 pixel values per step.

We implemented a single-layer, unidirectional LSTM followed by a fully connected classifier:

- **Input Representation:** Remove the channel dimension using `squeeze(1)` to obtain a row sequence of shape $[B, 28, 28]$.
- **LSTM Layer:** `LSTM(input_size=28, hidden_size=2048, batch_first=True)` produces an output of shape $[B, 28, 2048]$.
- **Hidden Representation:** Select the final timestep to obtain a representation of shape $[B, 2048]$ by $[:, -1, :]$.
- **Classifier:** `Linear(2048, 10)` produces class logits of shape $[B, 10]$.

The LSTM has **17,043,466 trainable parameters**, including 17,022,976 parameters in the recurrent layer and 20,490 in the classifier. PyTorch's LSTM uses separate input-to-hidden and hidden-to-hidden bias vectors, giving the parameter formula:

$$P_{\text{total}} = 4H(I + H + 2) + (HK + K) \tag{2.6}$$

where $I = 28$ is the input size, $H = 2048$ is the hidden dimension, and $K = 10$ is the number of classes.

The LSTM uses input, forget, and output gates to control information flow through its cell and hidden states, allowing it to retain information across image rows. However, sequential row processing limits parallelization and does not explicitly capture local two-dimensional spatial relationships as CNNs do.

2.6 Transformer

Unlike CNNs and LSTMs, our Transformer uses self-attention to learn relationships between image patches. Each Fashion-MNIST image of shape $[B, 1, 28, 28]$ is divided into 49 non-overlapping patches of size 4×4. We implemented a compact Vision Transformer (ViT) with the following architecture:

- **Patch Embedding:** `Conv2d(1, 32, kernel_size=4, stride=4)` projects each patch into a 32-dimensional embedding, producing $[B, 49, 32]$.
- **Token and Positional Embedding:** A learnable classification token (CLS) is prepended, and learnable positional embeddings are added to preserve spatial information. Dropout(0.1) is applied, producing $[B, 50, 32]$.
- **Transformer Encoder:** Three encoder blocks, each containing pre-layer normalization, multi-head self-attention with 4 heads (8 dimensions per head), residual connections, and a feed-forward network `Linear(32, 64)` $\rightarrow$ GELU $\rightarrow$ Dropout $\rightarrow$ `Linear(64, 32)`. The output remains $[B, 50, 32]$.
- **Classifier:** Select the final CLS representation $[B, 32]$ $\rightarrow$ LayerNorm $\rightarrow$ Linear(32, 10), producing class logits $[B, 10]$.

Self-attention computes relationships between tokens through Query (Q), Key (K), and Value (V) projections:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{2.7}$$

where $d_k = 32/4 = 8$. Each attention head computes a 50×50 attention matrix, allowing every token to incorporate information from other image patches.

The Transformer has **28,202 trainable parameters**.

Self-attention allows the model to capture relationships between distant image regions and process tokens in parallel. However, unlike CNNs, the Transformer has weaker built-in spatial inductive biases.

2.7 Training Setup

2.7.1 Training Configuration

The dropout configuration used by each architecture is summarized in Table 2.2.

Table 2.1: Main Training Configuration

Hyperparameter	**Configuration**
Dataset	Fashion-MNIST
Training samples	54,000
Validation samples	6,000
Test samples	10,000
Batch size	200
Learning rate	0.001
Optimizer	Adam
Loss function	`CrossEntropyLoss`
Random seed	666
Main training duration	10 epochs
Checkpoint criterion	Highest validation macro-F1
Learning-rate scheduler	None
Early stopping	None
Data augmentation	None
Mixed-precision training	Not implemented

Table 2.2: Dropout Configuration by Model

Model	**Dropout probability**
Linear	None
MLP	0.20
CNN	0.35
LSTM	None explicitly configured
Transformer	0.10

2.7.2 Checkpoint Selection

The training pipeline saves a model checkpoint after every epoch. The best checkpoint is updated whenever validation macro-F1 improves. After training, the selected checkpoint is loaded for test evaluation.

Using validation macro-F1 as the checkpoint criterion ensures that model selection considers performance across all classes rather than relying exclusively on overall accuracy.

2.7.3 Hyperparameter Selection

Hyperparameters were evaluated experimentally during model development.

2.8 Experimental Design

We designed two main experiments to compare the performance of different neural network architectures and investigate the effect of training duration.

Experiment 1: Architecture Comparison. The first experiment compares five models: Linear, MLP, CNN, LSTM, and Transformer. We hypothesized that architectures capable of learning spatial or sequential relationships would perform better than simpler models on Fashion-MNIST.

- **Changed Factor:** Model architecture.
- **Fixed Factors:** Fashion-MNIST split (54,000 training, 6,000 validation, and 10,000 test images), random seed 666, batch size 200, Adam optimizer with learning rate 0.001, cross-entropy loss, identical input standardization, and 10 training epochs.
- **Evaluation Metrics:** Test accuracy, macro-F1, trainable parameter count, training time, and inference time.
- **Decision Criteria:** Compare classification performance alongside computational cost and model complexity, rather than selecting a model based on accuracy alone.

All models use the same data partitions and evaluation procedure. The checkpoint with the highest validation macro-F1 is selected for final test evaluation. Model-specific architectures and regularization settings remain different, so this experiment compares practical configurations rather than isolating architecture under equal parameter budgets.

Experiment 2: Attention vs. Convolution. The third experiment compares the Vision Transformer (ViT), which uses 4×4 image patches and self-attention, with the Convolutional Neural Network (CNN), which uses local convolutional filters. We hypothesized



that the CNN's spatial locality inductive bias would provide an advantage on Fashion-MNIST, while the Transformer would initially underperform but improve with additional training epochs.

- **Changed Factor:** Model architecture and input representation (CNN with 3×3 convolutional filters versus ViT with 4×4 patches and self-attention).
- **Fixed Factors:** Fashion-MNIST dataset, training/validation/test split, preprocessing, random seed, optimizer, learning rate, batch size, loss function, and initial training duration of 10 epochs.
- **Evaluation Metrics:** Test accuracy, test macro-F1, trainable parameter count, training time, and inference time.
- **Decision Criteria:** Determine whether the CNN outperforms the ViT after 10 epochs and whether extending ViT training to 30 epochs reduces the performance gap.

  Experiment 3: Transformer Training Duration. The second experiment investigates whether additional training improves the compact Transformer's performance. We hypothesized that the Transformer could benefit from more optimization epochs because its validation performance was still improving during the initial experiment.

  - **Changed Factor:** Training duration (10 versus 30 epochs).
  - **Fixed Factors:** Transformer architecture, data split, preprocessing, random seed, optimizer, learning rate, batch size, and loss function.
  - **Evaluation Metrics:** Validation macro-F1, test accuracy, test macro-F1, and training time.
  - **Decision Criteria:** Determine whether longer training provides meaningful classification improvements relative to the additional computational cost.

3 Implementation Results

3.1 Training Behavior

The analysis focuses on convergence, generalization, overfitting, checkpoint selection. The main comparison uses ten training epochs for each architecture. The best checkpoint is selected according to the highest validation macro-F1 score.

3.1.1 Linear Classifier

The Linear Classifier showed rapid improvement during the early epochs. Training loss decreased from approximately 0.6156 at epoch 1 to 0.3958 at epoch 10, while validation loss decreased from 0.4832 to 0.4052.

The highest validation macro-F1 of 0.8590 was achieved at epoch 6. Although training loss continued decreasing afterward, validation macro-F1 fluctuated around a relatively stable value.

The limited representational capacity of a single linear transformation may restrict its ability to capture complex relationships between image pixels.

3.1.2 Multilayer Perceptron

The MLP demonstrated consistent improvements throughout the ten training epochs. Training loss decreased from 0.5302 to 0.2521, while validation loss decreased from 0.4117 to 0.2878. Validation macro-F1 increased from 0.8502 at epoch 1 to 0.8989 at epoch 10.

The best checkpoint was therefore selected at epoch 10. The continued improvement suggests that the model had not clearly reached its performance limit within ten epochs.

3.1.3 Convolutional Neural Network

The CNN demonstrated rapid convergence and achieved the highest validation macro-F1 among the five architectures. Validation macro-F1 increased from approximately 0.9047 at epoch 1 to 0.9389 at epoch 10.

However, the loss curves revealed signs of overfitting. Training loss decreased from 0.4610 at epoch 1 to 0.0646 at epoch 10. Validation loss initially decreased to its minimum of approximately 0.1875 at epoch 5, before increasing to 0.2564 at epoch 10.

The increasing difference between training and validation loss indicates that the model became more confident on training examples without producing a corresponding improvement in validation loss.

3.1.4 Long Short-Term Memory

The LSTM exhibited progressive improvement throughout training. Validation macro-F1 increased from 0.8142 at epoch 1 to a maximum of 0.9100 at epoch 9. Training loss decreased from approximately 0.8475 to 0.1951 across ten epochs.

Validation loss reached its lowest value of approximately 0.2554 at epoch 9, before increasing slightly to 0.2621 at epoch 10. Similarly, validation macro-F1 decreased

from 0.9100 to 0.9064 between epochs 9 and 10. These observations indicate that the model's generalization performance did not continue improving monotonically until the final epoch. Therefore, epoch 9 was selected as the best checkpoint.

The LSTM achieved strong classification performance but required substantially more parameters than the other evaluated architectures.

3.1.5 Transformer

The Transformer began training with lower classification performance than the other architectures. Its initial training loss was approximately 1.2064, compared with a validation loss of 0.6367.

During training, validation macro-F1 increased from 0.7586 at epoch 1 to a maximum of 0.8636 at epoch 9. Training loss decreased to 0.4134 by epoch 10, while validation loss decreased to 0.3731. Although the best validation macro-F1 occurred at epoch 9, validation loss continued decreasing at epoch 10.

This suggests that the compact Transformer may benefit from additional training. The separate thirty-epoch experiment supports this interpretation, achieving a maximum validation macro-F1 of 0.8902. However, additional training also increases computational cost.

3.2 Quantitative Results

3.2.1 Main Model Comparison

Table 3.1: Main Model Comparison

Model	Test Accuracy (%)	Test Macro-F1	Trainable Parameters
Linear	83.91	0.8371	7,850
MLP	88.12	0.8816	203,530
CNN	93.19	0.9316	3,397,578
LSTM	90.04	0.9004	17,043,466
Transformer	84.81	0.8492	28,202

The CNN achieved the highest test accuracy of 93.08% and macro-F1 of 0.9310 among the five selected configurations.

The LSTM achieved the second-highest accuracy of 90.04%, but required approximately 17.04 million trainable parameters.

The MLP achieved 88.12% accuracy with substantially fewer parameters than either the CNN or LSTM.

The Transformer achieved 84.81% accuracy using only 28,202 trainable parameters.

The Linear Classifier had the smallest parameter count but also the lowest classification accuracy.

These findings demonstrate that the number of trainable parameters does not independently determine model performance.

3.2.2 Training and Inference Time

It is not absolutely true for every machine testing this experiment, time vary base on your resources

Table 3.2: Recorded Training and Inference Time

Model	**Training time (s)**	**Inference time (s)**
Linear	58.90	0.729
MLP	92.19	0.761
CNN	173.23	0.950
LSTM	185.49–237.83*	1.936–2.030*
Transformer	75.24	0.794

3.2.3 Additional Transformer Experiment

A separate experiment extended Transformer training from ten to thirty epochs while preserving the model architecture.

The saved histories contain identical metric values for the first ten epochs, indicating that the thirty-epoch history extends the recorded ten-epoch trajectory.

3.3 Qualitative Results

When we ran our evaluation script (`src/evaluate.py`) on the test set, we noticed several interesting patterns:

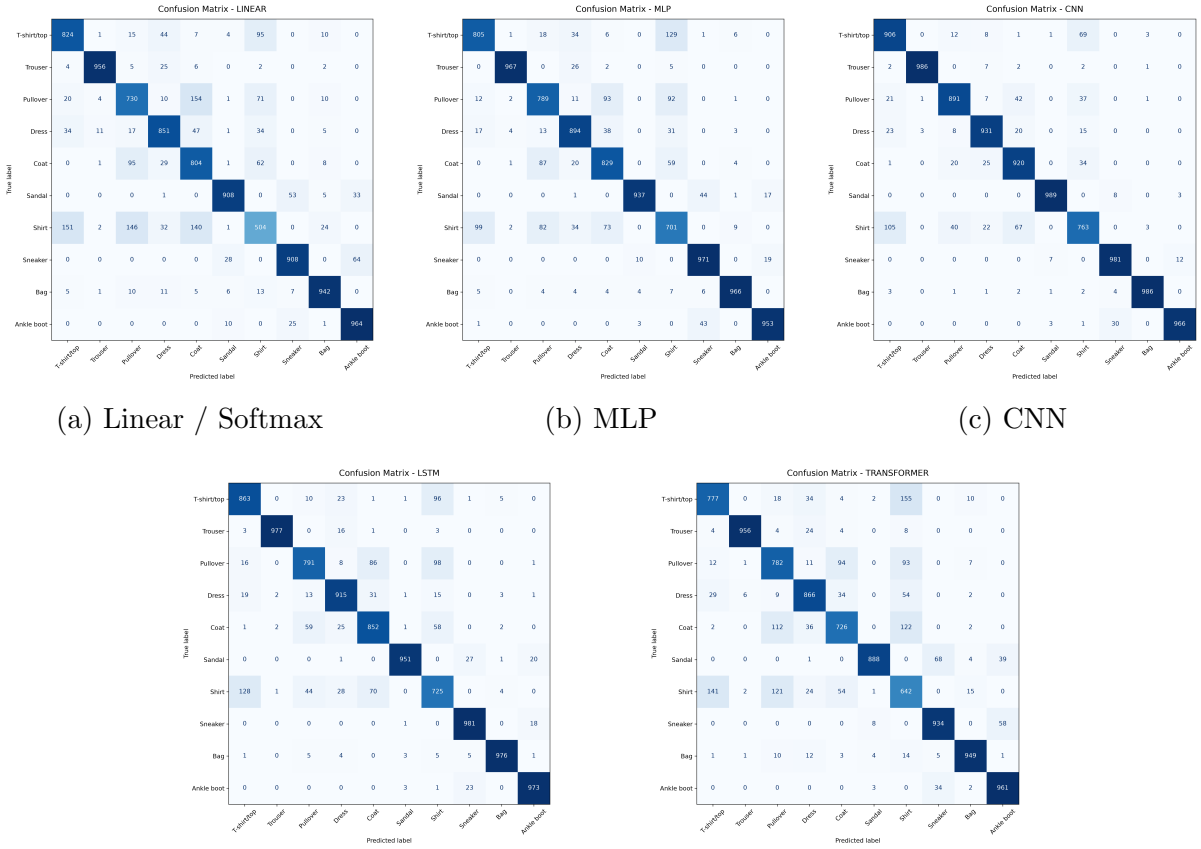
1. **Confusion Matrices:** Comparing the confusion matrices, CNN has the clearest diagonal line across all 10 classes, meaning it makes the fewest mistakes. For all models, the **Shirt** class is the hardest to predict, and it often gets confused with

Table 3.3: Transformer Training Duration Comparison

Metric	**10 epochs**	**30 epochs**
Best validation macro-F1	0.8636	0.8902
Test accuracy (%)	84.81	87.33
Test macro-F1	0.8492	0.8738
Trainable parameters	28,202	28,202
Training time (s)	75.24	222.50

T-shirt/top, *Pullover*, or *Coat*. On the other hand, shoes (*Sandal, Sneaker, Ankle boot*) and *Trouser* have very high accuracy ($> 96 - 98\%$) in all models.

2. **Looking at Misclassified Images:** When we inspected the images that models got wrong, most of them were shirts with no visible collar or buttons (which look just like t-shirts), or long coats that look almost like dresses at 28×28 resolution.



(a) Linear / Softmax (b) MLP (c) CNN

(d) LSTM (e) Transformer

Figure 3.1: Normalized confusion matrices across all five evaluated architectures on the Fashion-MNIST test split.



3.4 Comparison and Discussion

We compared how the design of each model affects its performance and computational efficiency:

1. **Why is CNN the best model (93.19% Test Acc)?**
   Images are made of 2D pixels where nearby pixels are related. CNN uses small 3×3 sliding filters and shares weights across the whole image, which matches how images work in real life. This allows the CNN to learn edges, textures, and clothing outlines easily, even if the object shifts slightly. Also, even though our CNN has 3.39M parameters, over 90% of them are just in the first linear layer after flattening; the convolutional layers themselves take fewer than 200k parameters.

2. **The problem with LSTM (17 million parameters):**
   Although our LSTM achieved 90.04% test accuracy, it needs a huge number of parameters (17,043,466 parameters, 5 times more than CNN) and takes 1.94 s to evaluate the test set (2.5 times slower than CNN). Reading an image row-by-row like a sentence is unnatural for 2D clothes because it ignores horizontal relationships and forces the computer to process row after row sequentially, which does not take full advantage of GPU parallel computing.

3. **Vision Transformer: very small but needs more epochs:**
   Our ViT has only 28,202 parameters (around 120 times smaller than CNN) and runs very fast (0.79s latency). After 10 epochs, it gets 84.81%, but when we trained it for 30 epochs, its accuracy jumped to 87.81% (nearly reaching MLP). Because Transformers do not have built-in assumptions about 2D image locality, they have to learn spatial relationships completely from the data, which means they need more epochs to train well.

4. **Which model to choose in real life?**
   If we want the highest accuracy and fast inference, *CNN* is clearly the best choice (93.19% accuracy, 0.78s inference). But if we want to run our model on small edge devices or microcontrollers with very little memory, *ViT* (28k parameters) or *MLP* (200k parameters) are good options because they take very little memory while still achieving around 88% accuracy.

3.5 Error Analysis

Looking closely at the prediction outputs from evaluate.py, we noticed that the errors made by all five models are definitely not random. Instead, our error cases clearly group into three main patterns based on visual shape.

The most common errors happen within the upper-body clothing group (shirt, T-shirt/top, pullover, and coat). Across the five models, around 105 to 151 shirts are misclassified as T-shirts, 67 to 140 shirts are predicted as coats, and 42 to 154 pullovers get confused with coats. This makes a lot of sense because all four items share almost the exact same outline. When the images are downscaled to just 28 x 28 grayscale pixels, small visual cues like collar folds, button lines, or neckline cuts become completely blurry, making it really hard for any model to tell them apart.

We also saw some occasional mix-ups among footwear classes. Specifically, models confuse sneaker with ankle boot (about 25 to 45 errors), and sandal with sneaker (around 10 to 20 errors). In our dataset, some low-cut ankle boots have thick rubber soles that look quite similar to high-top sneakers. Similarly, sandals with thick straps look almost like flat sneakers when viewed from the side at such low resolution.

The third pattern is a smaller cluster of errors between dress and coat (roughly 15 to 35 samples). Long winter coats drop straight down past the knees, giving them a 2D silhouette that looks almost identical to long dresses against a plain background.

Out of all 10 classes, shirt is clearly the hardest one for every model to learn. It consistently has the lowest recall across the board: only 50.4% on Linear, 64.2% on ViT (10 epochs), 70.1% on MLP, 72.5% on LSTM, and 76.3% on CNN. In practice, the only real difference between a collared shirt and a simple T-shirt is just a tiny patch of pixels around the neck. Because the dataset lacks color and high resolution, these subtle details get easily washed out by pooling layers.

3.6 Limitations and Conclusion

3.6.1 Dataset Limitations

The experiments were conducted using Fashion-MNIST, a relatively small and controlled image classification dataset containing grayscale images of size 28×28 pixels.

Although this dataset is suitable for comparing architectural families, its limited resolution and visual complexity may not represent more challenging image classification tasks.

Therefore, the results should not be directly generalized to larger, higher-resolution,



or more complex datasets without additional evaluation.

3.6.2 Preprocessing Trade-off: Evaluation Consistency vs. Distribution Adaptation

The preprocessing pipeline computes standardization statistics exclusively from the training partition and applies these statistics to training, validation, and test images.

This strategy prioritizes consistent evaluation and data leakage prevention. Using fixed training statistics ensures that all partitions undergo the same normalization procedure and prevents validation or test distribution information from influencing the preprocessing configuration.

However, fixed standardization may become less appropriate when new input images follow a different distribution. The current implementation therefore prioritizes evaluation integrity and reproducibility over adaptation to potentially shifted input distributions.

3.6.3 Architectural and Computational Limitations

The evaluated architectures differ considerably in their model capacity and computational requirements. For example, the LSTM contains approximately 17.04 million trainable parameters, while the Transformer contains only 28,202.

The experiments therefore do not isolate architectural effects under matched parameter budgets. Additionally, the primary comparison uses a fixed duration of ten epochs. This provides a common training-duration setting, but not necessarily an equal optimization opportunity or computational budget for every architecture.

3.6.4 Experimental and Reproducibility Limitations

Several limitations affect the reliability and reproducibility of the experiments.

First, the main comparison uses one fixed seed, not a random seed for every trial.

Second, training-time measurements include evaluation and checkpoint-saving overhead, making them unsuitable as isolated measurements of computational training cost.

3.6.5 Conclusion

This assignment investigated five neural network architectures for Fashion-MNIST image classification using a shared data preparation and evaluation framework.

The experimental results demonstrate differences in classification performance, model capacity, convergence behavior, and computational requirements.

Among the selected ten-epoch configurations, the CNN achieved the highest test accuracy of 93.08% and macro-F1 of 0.9310.

The LSTM achieved 90.04% accuracy but required substantially more trainable parameters.

The compact Transformer achieved lower accuracy within the ten-epoch comparison, although its performance improved in the additional thirty-epoch experiment.

Overall, the findings suggest that architectural, bias, input, parameter capacity, and training budget are important factors when selecting deep learning models for image classification.

4 AI Usage Disclosure

In compliance with Section 5 of the Course Project Handbook, Group RTX5090 transparently discloses all generative AI tools used throughout Assignment 1. Rather than generating core scientific results, AI tools were used exclusively for code syntax support, library usage guidance, and writing assistance as detailed below:

Gemini 3.8 Flash (Google) — Used by Cao Duc Thanh:

- Purpose & Prompts: Asked for syntax guidance on Matplotlib subplots to plot the three convergence curves (Loss, Accuracy, Macro-F1) together, and requested helper logic to traverse confusion matrix off-diagonal elements to identify top misclassified pairs. In addition, consulted Gemini to debug tensor reshaping issues in the LSTM network, implement the compact Vision Transformer architecture based on the Dive into Deep Learning (D2L) textbook, and assist in refining the LaTeX report into clear, concise student prose without excessive jargon.

- Affected Files: plot_curves.py, src/evaluate.py, src/models/lstm.py, src/models/transformer.py, and assignment 1 report.

- Student Verification: The student checked and executed the models locally on GPU, verified that training converged normally (accuracy reached $\sim 90\%$ for LSTM and $\sim 85 - 88\%$ for ViT), verified the confusion matrices and error counts against test outputs, and ensured the LaTeX document compiled cleanly.



References

A Appendix